

IAL – 6. přednáška



Vyhledávací tabulky III.

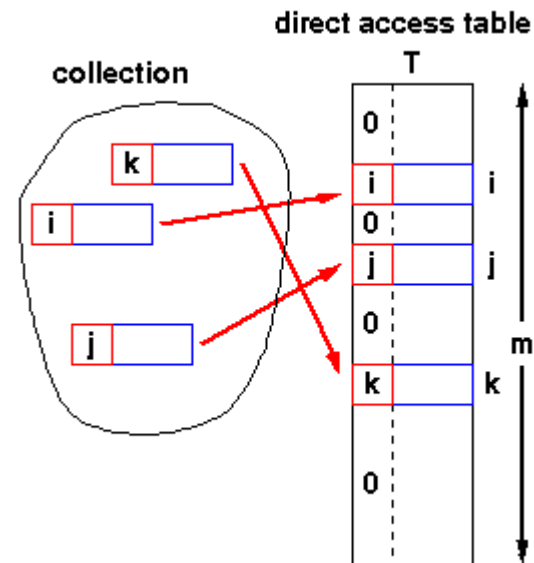
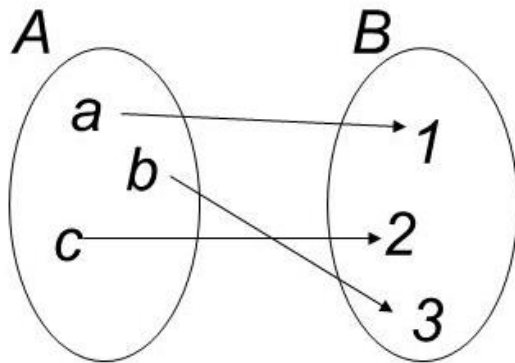
31. října a 1. listopadu 2023

Obsah přednášky

- Tabulky s přímým přístupem (TPP)
- Mapovací funkce
- Tabulky s rozptýlenými položkami (TRP)
 - Princip index-sekvenčního vyhledávání
 - Explicitní a implicitní zřetězení synonym
- Zhodnocení metod vyhledávání

Tabulka s přímým přístupem (TPP)

- Implementace vyhledávací tabulky polem, ve které jsou klíče mapovány na indexy pole:
 - Ideální struktura z pohledu vyhledávání
 - Bohužel obvykle nerealizovatelná
- Vyžaduje **vzájemně jednoznačné zobrazení (bijekce)** mapující každý prvek množiny klíčů K do množiny indexů pole H (sousedních adres v paměti).



TPP: příklad

- Uvažujme množinu klíčů danou intervalem **<0..999>** a tabulku reprezentovanou polem **T[0..999]**.
- Pak můžeme použít zobrazení, které mapuje klíč **K** na pozici **T[K]**.
- Klíče typu **int** mohou takto být snadno mapovány na indexy, proto se používají v řadě případů.
- **Pozn.:** Všimněte si, že rozsah pole zde odpovídá rozsahu hodnot klíče (počtu všech možných hodnot klíče). Tento rozsah však v praxi bývá často **neúnosně velký**.

TPP: implementace

- Každému prvku pole pořídíme booleovskou složku `busy`, která určuje, zda v tabulce prvek s daným klíčem existuje nebo neexistuje (je nebo není).
- Při inicializaci se pro všechny prvky tabulky nastaví `busy ← false`.
- **Vyhledávání:** spočívá v **přímém zjištění**, zda na pozici klíče (indexu) dané tabulky je nebo není obsazeno.
- Časová složitost přístupu v TPP: $\Theta(1)$
- **Obtíž:** nalezení **vhodné mapovací funkce**.

TPP: příklad 2

- Předpokládejte, že chceme ukládat informace o studentech. Každému studentovi je přiřazeno jedinečné sedmimístné identifikační číslo, které může sloužit jako index do pole realizující vyhledávací tabulku:

0		
:	:	:
12345	andy	81.5
:	:	:
33333	betty	90
:	:	:
56789	david	56.8
:	:	:
:	:	:
9908080	bill	49
:	:	:
9999999		

- Pro uložení informací např. o 1 000 studentech bychom potřebovali pole o 10 000 000 prvcích (využití paměti 0,1 %)! Identifikační číslo tedy není vhodné použít přímo, ale přes **mapovací funkci**.

Mapovací funkce

- Nalezení vzájemně jednoznačného zobrazení (mapovací funkce) je velmi obtížné $\Rightarrow$ je potřeba počítat s tím, že běžná mapovací funkce může **různým klíčům přiřadit stejnou hodnotu** (stejně místo v paměti).
 - **Kolize** – dva různé klíče jsou namapovány do stejného místa.
 - **Synonyma** – dva nebo více klíčů, které jsou namapovány do téhož místa.
- $\Rightarrow$ **tabulka s rozptýlenými položkami**
(TRP, hashovací tabulka)

Mapovací funkce

- **Obtíž:** Nalezení vhodné **mapovací funkce – proč?**
 - Například pro 31 prvků, které se mají zobrazit do 41 prvkové množiny, existuje 41^{31} tj. cca 10^{50} mapovacích funkcí.
 - Přitom jen $(41!/10!)$ tj. cca 10^{43} z nich jsou vzájemně jednoznačné.
 - Poměr *ideálních* funkcí, které žádné dva klíče nemapují na stejné místo, *ku všem možným* je zde asi **1:10 000 000**.

- **Příklad:** Paradox společných narozenin
 - Je dobrá naděje, že mezi 23 osobami, které se sejdou ve společnosti, se najdou dvě osoby, které mají narozeniny ve stejný den.
 - Jinými slovy: Vybereme-li náhodně funkci, která mapuje 23 klíčů do tabulky o 365 prvcích, je pravděpodobnost, že se žádné dva klíče nenamapují do stejného místa, 49,3 %.

Mapovací funkce

- Nechť je dáno mapovací pole s rozsahem $[0...N]$ nebo $[1...N]$.
- Mapovací funkce **transformuje klíč na index** v daném rozsahu.
- Typicky lze rozdělit do dvou etap:
 - převod klíče na přirozené číslo ($N > 0$),
 - převod přirozeného čísla na hodnotu spadající do intervalu (nejčastěji s použitím operace modulo).

Mapovací funkce – požadavky

- Determinismus
 - Pro daný klíč vrátí vždy stejnou hodnotu.
- Rovnoměrné (uniformní) rozložení
 - Na každé místo se mapuje přibližně stejně velké množství klíčů.
- Využití celých vstupních dat
- Vyhnutí se kolizím podobných klíčů
 - V praxi bývá řada klíčů velice podobných.
- Rychlý výpočet
- Neexistuje obecné pravidlo, jak nalézt nejvhodnější mapovací (rozptylovací) funkci.

Mapovací funkce: příklady

- ❑ PJW (Peter J. Weinberger)
- ❑ ELF Hash (UNIX)
- ❑ BKDR (Brian Kernighan, Dennis Ritchie)
- ❑ DEK (Donald E. Knuth)
- ❑ DJB (Daniel J. Bernstein)
- ❑ ...

Ukázka mapovací funkce – BKDR

```
unsigned int BKDRHash(char* str, unsigned int length)
{
    unsigned int seed = 131;
    unsigned int hash = 0;
    unsigned int i = 0;
    for (i = 0; i < length; str++, i++)
    {
        hash = (hash * seed) + (*str);
    }
    return hash;
}
```

Ukázka mapovací funkce – DJB

```
unsigned long DJBHash(unsigned char *str)
{
    unsigned long hash = 5381;
    int c;
    while (c = *str++)
        hash = ((hash << 5) + hash) + c;
    return hash;
}
```

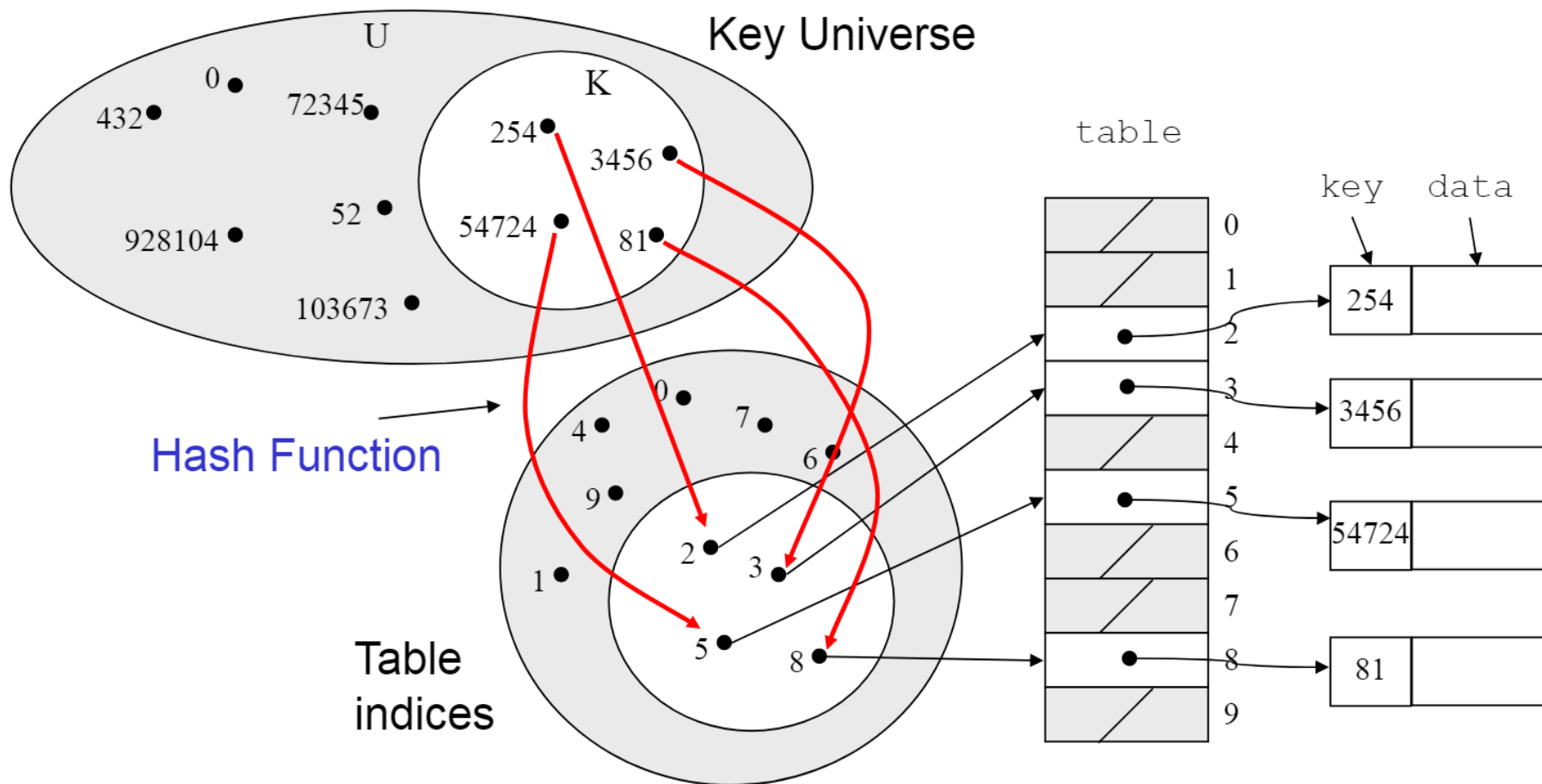
Převod hodnot do hranic pole

- Necht' **K** je celé číslo větší než nula.
- Funkce **$h_0(K) = K \bmod (Max+1)$** získá hodnoty z intervalu $\langle 0..Max \rangle$.
- Funkce **$h_1(K) = K \bmod Max + 1$** získá hodnoty z intervalu $\langle 1..Max \rangle$.

Tabulka s rozptýlenými položkami

- Tabulka s rozptýlenými položkami (TRP) sestává:
 - z mapovacího prostoru (pole) a
 - ze seznamů synonym.
- Seznam synonym (i prázdný) začíná na každém prvku mapovacího pole.
 - **Explicitní zřetězení** – adresa následníka je obsažena v jeho předchůdci (zřetězení záznamů).
 - **Implicitní zřetězení** – adresa následníka se získá pomocí funkce z adresy předchůdce (otevřená adresace).

Ukázka tabulky s rozptýlenými položkami



Vyhledávání v TRP

□ Princip vyhledávání v TRP spočívá ve **dvou krocích**:

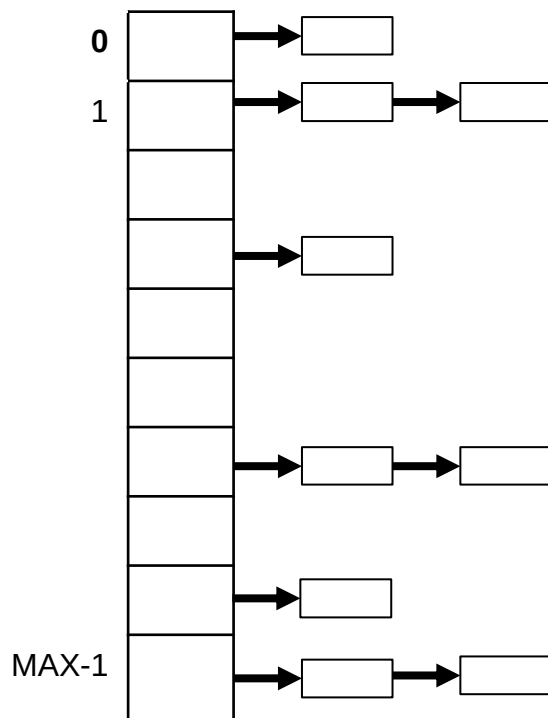
1. Nalezení indexu prvku v poli k danému klíči pomocí mapovací funkce (na tomto indexu začíná seznam synonym, které se namapovaly do tohoto místa).
2. Sekvenční průchod tímto seznamem synonym (vyhledáváme položku s daným klíčem).

⇒ Vyhledávání v TRP má **index-sekvenční** charakter.

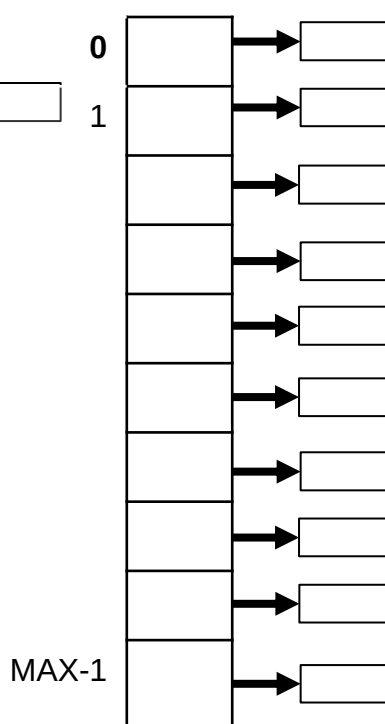
TRP s explicitním zřetězením synonym

- Seznam synonym je obvykle realizován jako lineární seznam.
- Maximální doba vyhledávání je pak dána délkou nejdelšího seznamu synonym – $O(n)$.

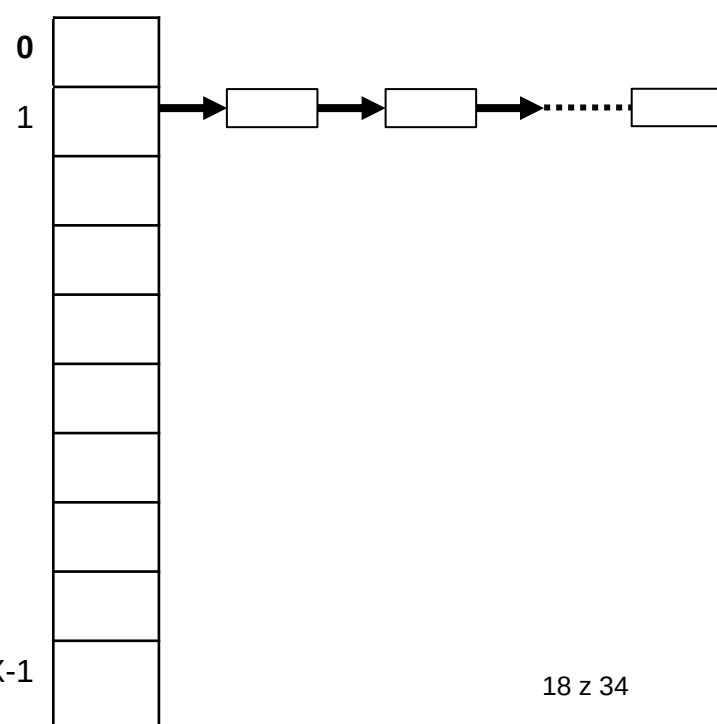
běžný stav



optimální stav



nejhorší stav

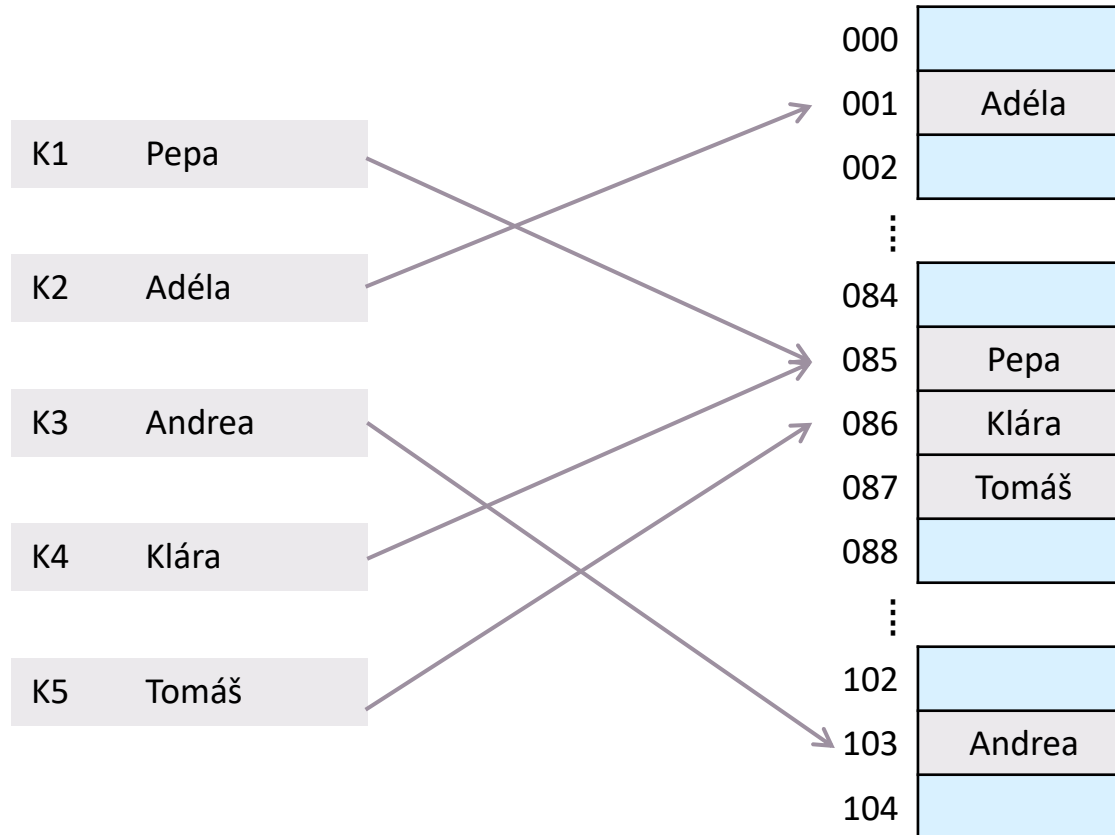


Alternativní zřetězení synonym

- ❑ Místo lineárních seznamů pro uložení synonym lze použít **vyvažované binární vyhledávací stromy**.
- ❑ Pak je časová složitost v nejhorším případě **$O(\log_2 n)$** .

TRP s implicitním zřetězením synonym

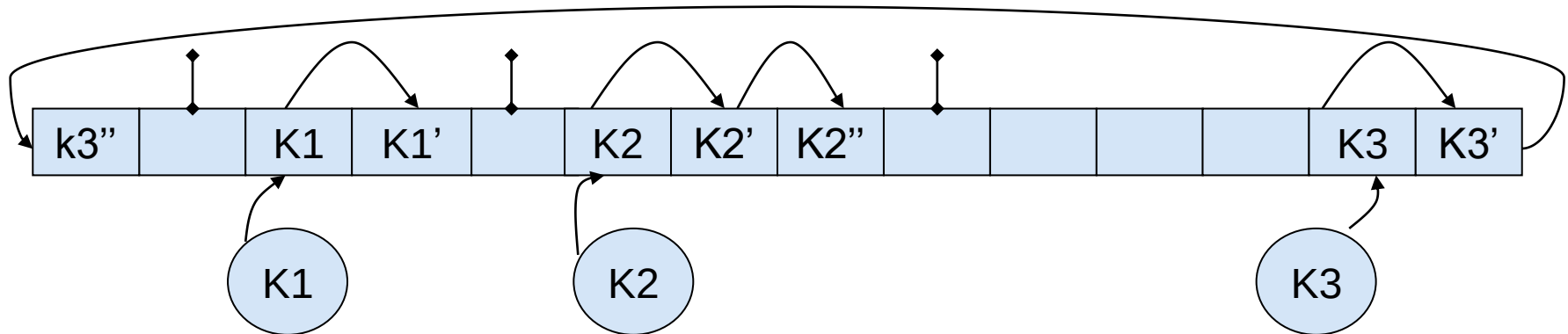
- TRP implementovaná polem, ve kterém jsou uloženy jak první prvky seznamů synonym, tak jejich další položky



TRP s implicitním zřetězením synonym

- TRP s otevřenou adresací
- Pro přístup k synonymům existují různé metody pro určení kroku:
 - Lineární: $h(k, i) = (h(k) + C*i) \% (Max+1)$
 - Kvadratická: $h(k, i) = (h(k) + C_1*i + C_2*i^2) \% (Max+1)$
 - S dvojí rozptylovací funkcí:
 $h(k, i) = (h_1(k) + h_2(k)*i) \% (Max+1)$
 - *Legenda:*
 - $i = 0, 1, 2, \dots$ – pokusy o vložení
 - C, C_1, C_2 – konstanty
 - $Max+1$ – velikost pole

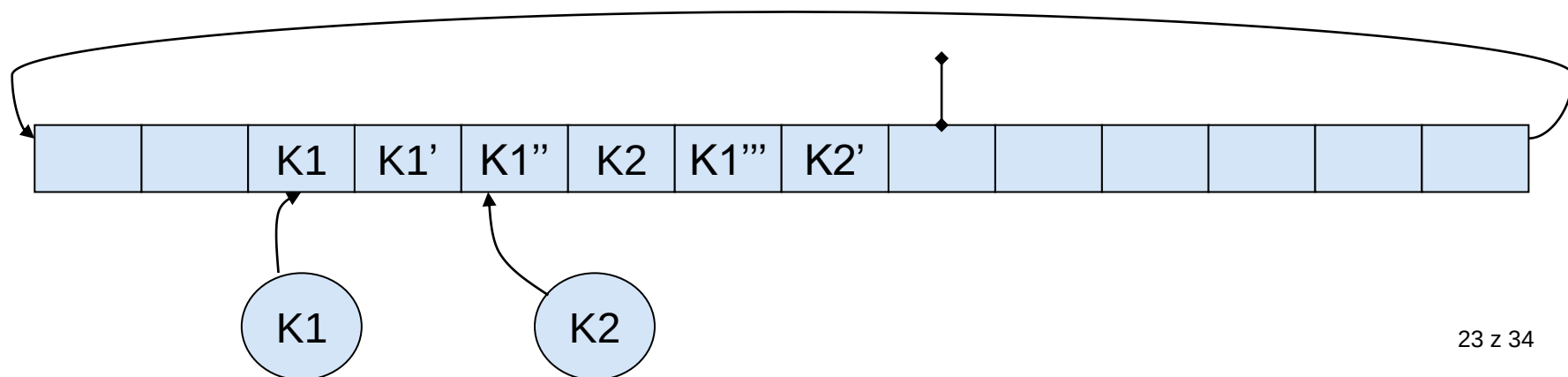
Implicitní zřetězení s pevným krokem



- ❑ Krok = 1: $a(i+1) = a(i) + 1$
- ❑ Konec seznamu synonym je dán **prvním volným prvkem**, který se najde se zadaným krokem.
- ❑ Nové synonymum se **vloží** na první volné místo (na konec seznamu).
- ❑ Tabulka (pole) musí obsahovat **alespoň jeden volný prvek**. Efektivní kapacita je o 1 menší než počet položek.
- ❑ Tabulka je implementovaná **kruhovým polem**.

Překrývání seznamů synonym

- Necht' jsou již do tabulky vloženy klíče K1, K1' a K1''.
- Následně se klíč K2 namapoval do položky, která je obsazena (je tam klíč K1''). Klíč byl **uložen na první volné místo**. Další klíč K1''' se namapoval do položky K1. První volné místo pro klíč K1''' bylo nalezeno za klíčem K2. Klíč K2' se namapoval do položky K1''. První volné místo se našlo za klíčem K1'''.
- V této tabulce se **dva seznamy synonym překrývají**. Prvek K1'' je vstupním bodem seznamu synonym K2.



Zrušení položky v TRP

- **Zrušení** položky v TRP s explicitním zřetězením synonym je poměrně snadné (odpovídá zrušení prvku v lineárním seznamu případně v binárním vyhledávacím stromu).
- *K zamyšlení:*
Jak lze **zrušit** položku v TRP s implicitním zřetězením synonym?

Velikost rozptylovacího pole (1/2)

- Krok s hodnotou 1 má tendenci vytvářet *shluky* (angl. *cluster*).
- Výhodnější je **krok větší než 1**. Takový krok by ale měl mít možnost *navštívit* všechny položky pole. Kdyby pole mělo sudý počet prvků, pak sudý krok by z výchozího lichého indexu mapovaného prvku mohl navštívit pouze liché indexy.
 - Např. pole [1..10], krok 2, začátek na indexu 5 projde indexy: 5, 7, 9, 1, 3.
 - Krok 4 a začátek na indexu 7 projde indexy: 7, 1, 5, 9, 3.

Velikost rozptylovacího pole (2/2)

- ❑ Kdyby měl krok hodnotu prvočísla, které je nesoudělné s jakoukoli velikostí pole, pak by mohl postupně projít všemi prvky pole.
- ❑ Výhodnější ale je, aby **hodnotu prvočísla měla velikost mapovacího pole**. Pak jakýkoli krok dovolí projít všemi prvky mapovacího pole.
- ❑ Je vhodné dimenzovat velikost mapovacího pole TRP tak, aby bylo rovno prvočíslu.

TRP s dvojí rozptylovací funkcí

- ❑ Metoda s **dvojí rozptylovací** (hashovací) **funkcí**:
krok v rozptylovacím poli je určen za běhu
druhou rozptylovací funkcí.
- ❑ Nechť má rozptylovací pole rozsah $\langle 0..Max \rangle$, (kde hodnota $Max+1$ je prvočíslo) a nechť $KInt$ je klíč transformovaný na celou nezápornou hodnotu.
- ❑ **První rozptylovací funkce** vrací index z intervalu $\langle 0..Max \rangle$:
$$ind_0 = h_1(KInt) = KInt \bmod (Max+1)$$
- ❑ **Druhá rozptylovací funkce** vytváří krok z intervalu $\langle 1..Max \rangle$:
$$krok = h_2(KInt) = KInt \bmod Max + 1$$

Vyhledání – Search

□ Vyhledávací cyklus:

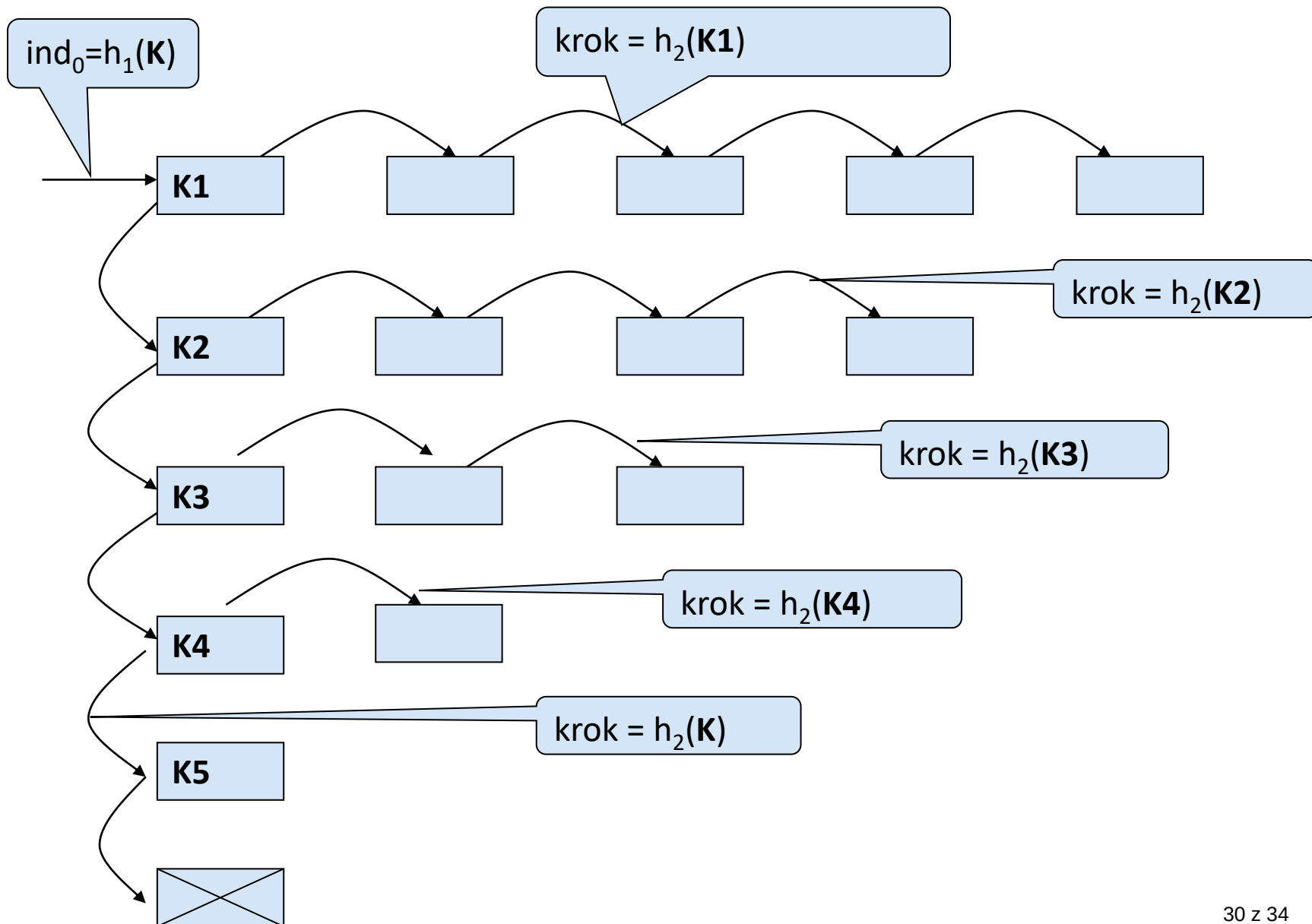
- Začíná na indexu získaném první rozptylovací funkcí.
- Končí úspěšně nalezením prvku s hledaným klíčem, neúspěšně při dosažení konce seznamu synonym (prázdným prvkem).
- Index následujícího prvku je dán přičtením kroku k aktuálnímu prvkem při respektování kruhovosti pole:

$$\text{ind}_{i+1} = (\text{ind}_i + \text{krok}) \bmod (\text{Max}+1)$$

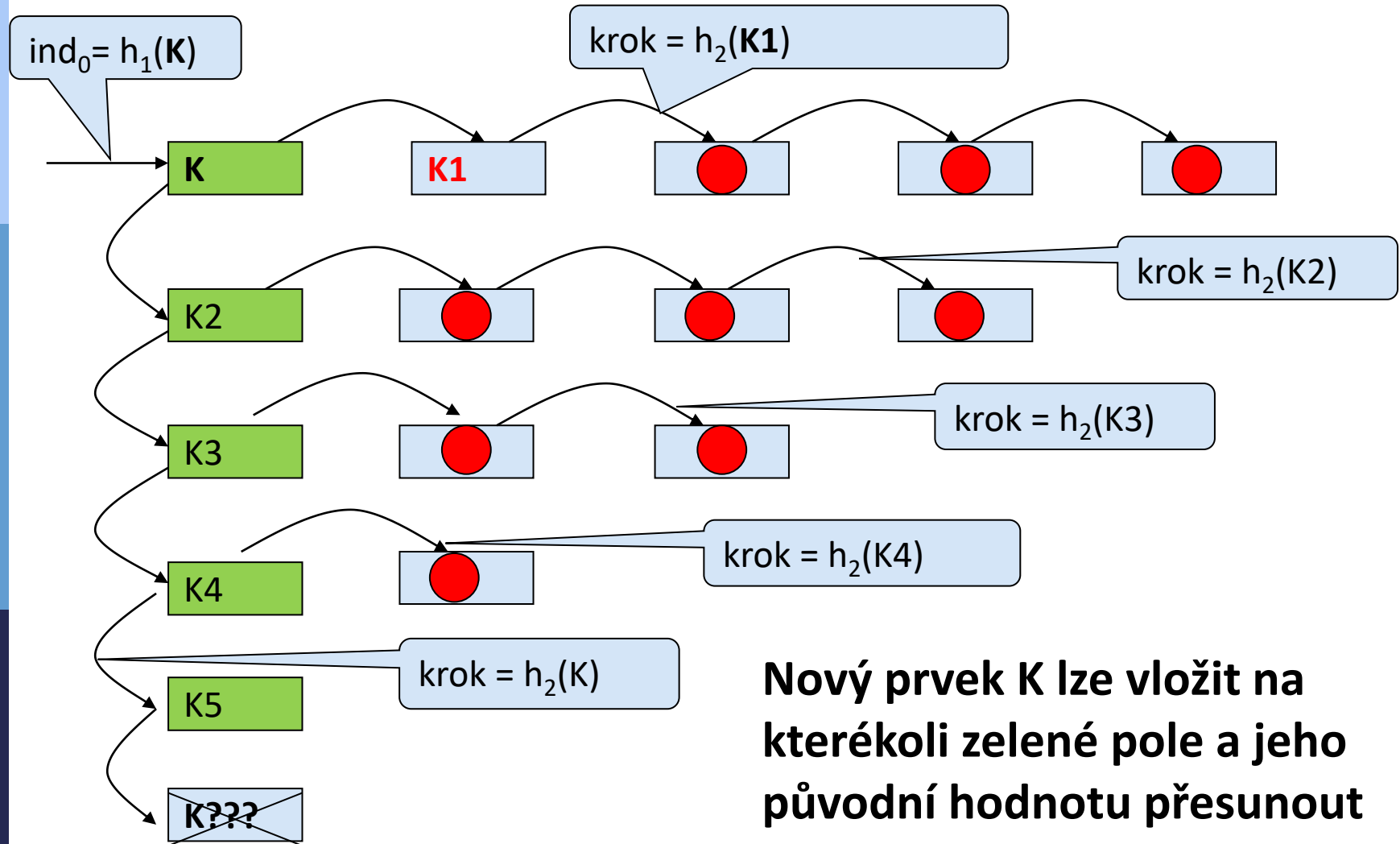
- Operace **Insert** vloží nový prvek na místo prvního prázdného prvku.

Brentova varianta

- ❑ Brentova varianta je **varianta metody TRP se dvěma rozptylovacími funkcemi**.
- ❑ Princip vyhledávání (Search) je shodný s TRP se dvěma rozptylovacími funkcemi.
- ❑ Brentova varianta je vhodná za podmínky, že počet případů úspěšného vyhledávání je častější než počet neúspěšného vyhledání s následným vkládáním.
- ❑ Brentova varianta provádí **při vkládání rekonfiguraci prvků** pole s cílem **investovat do vkládání** a získat lepší průměrnou dobu vyhledání.



Prvek K1 se přesune na první volné místo s krokem $h_2(K1)$
a na jeho místo se vloží prvek K.



**Nový prvek K lze vložit na
kterékoli zelené pole a jeho
původní hodnotu přesunout
na pole s červenou tečkou.**

Brentova varianta

- ❑ Normální metoda by nový klíč vložila na první volné místo s krokem $h_2(K)$ – zde po 6 krocích.
- ❑ **Brentova varianta** hledá první volné místo mezi červeně zakroužkovanými poli s krokem $h_2(K1)$, resp. $h_2(K2)$ atd. Na toto místo vloží prvek K1, resp. K2 atd. a na uvolněné místo vloží prvek K.
- ❑ Protože posun čelního prvku je menší než zde 5, je celková průměrná hodnota délky vyhledávání menší, než kdyby byl prvek K vložen na 6 pozici shora.

Hodnocení TRP s implicitním zřetězením

- ❑ **Operaci Delete** lze řešit pomocí *zaslepení* – vložením klíče, který nebude nikdy vyhledáván.
- ❑ TRP s implicitním zřetězením je vhodná v aplikacích, v nichž se **operace Delete nepoužívá příliš často**.
- ❑ **Maximální kapacita** TRP pro rozsah pole $<0..Max>$ je Max (o 1 menší než počet prvků pole) – alespoň jeden prvek musí zůstat jako *zarážka* vyhledávání.
- ❑ Pro efektivní použití TRP s implicitním zřetězením se pole tabulky dimenzuje tak, aby její maximální zaplnění (dané poměrem N_{akt}/Max) nebylo větší než cca 0,6-0,7.

Hodnocení metod vyhledávání

- Sekvenční vyhledávání n
- Binární vyhledávání v seřazeném poli $\log_2 n$
- Binární vyhledávací strom $\log_2 n$
 - při degradaci na seznam n
- Vyvažovaný BVS (např. AVL) $\log_2 n$
- TRP 1
 - při maximální kolizi klíčů n
 - při maximální kolizi a vyváženém stromu $\log_2 n$